



COMP 364/464: High-Performance Computing Week 05, Lecture 05



Agenda

1. C review
2. Shared Memory Parallelism: OpenMP
3. “Lab”

C Review

1. Compile pipeline
2. Processes and threads
3. Memory
4. Pointers

Compilation

Stages:

- Preprocess
- Compile
- Assemble
- Link

Output is an ELF executable



Compilation

Stages:

- Preprocess
 - Text processor, handles things like #include, #define, #ifdef
 - Outputs .i (just C with no more #)
 - Remember this for OMP



Compilation

Stages:

- Compile
 - Call a tool to translate C into human-readable assembly
 - Compiler choice and flags determine exactly what happens here
 - Outputs .s



Compilation

Stages:

- Assemble
 - Translate from assembly to machine code
 - Outputs .o files



Compilation

Stages:

- Link
 - Take many .o files and combine them where needed into an executable
 - Output is .out (ELF)
 - ELF: executable and linkable format
 - .text: code
 - .data: initialized globals
 - .rodata: read-only data
 - .bss: zero-initialized globals
 - Also symbol tables, strings, dynamic links, and debug (-g)



Understanding to check:

- What is the difference between declare and define?
- Which does the compiler care about?
- What compilation step cares about the other?

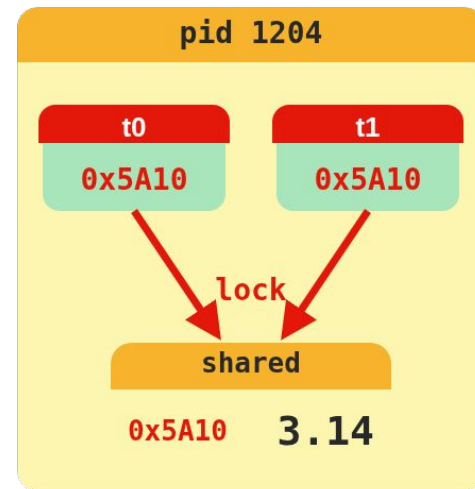
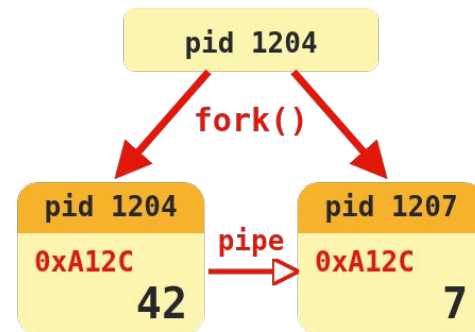
```
extern int count;          /* declaration: count exists somewhere */
int count = 0;             /* definition: storage for count */
double area(double r);    /* declaration (prototype) */
double area(double r) {   /* definition: has a body */
    return 3.14159 * r * r; }
```

Processes and Threads

- Process: running program, complete with address space
- Thread: lightweight process, more like execution path
- Processes make threads (shared memory parallelism)
- Threads share:
 - Code
 - Global and static variables
 - Heap
- Threads don't share:
 - Stacks
 - Registers, PCs, signal masks

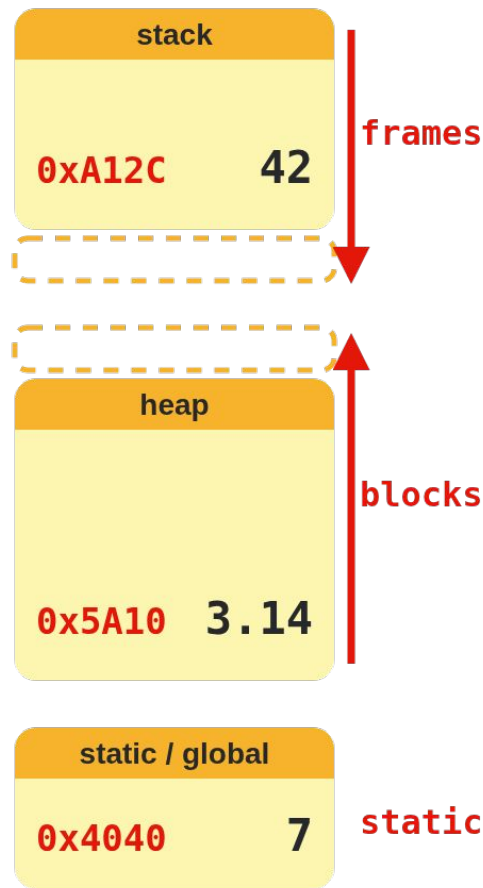
Processes and Threads

- Multiple processes:
 - `fork()`
 - No shared memory
 - Communication uses sockets, pipes, signals, etc
 - Switching is a process context switch
- Multiple threads:
 - Depends on how you create them (OpenMP, pthreads, etc.)
 - Memory is shared
 - Communication uses sentinels, locks, atomics, etc
 - Switching is nearly free --- instructions change, not address space



Memory

- Each process gets its own
- Each thread (lightweight process) gets some of its own
- Stack: locals, freed automatically when the function returns
- Heap: malloc/calloc/realloc, lives until you free it
- Static/global: lives for the whole program



Memory Functions

```
void *malloc(size_t size);
```

```
void *calloc(size_t nmemb, size_t size);
```

```
void free(void *ptr);
```

- Every allocation should have exactly one matching free
- Header: <stdlib.h>

Pointers

- Declared with (type) *

```
int * p;
```

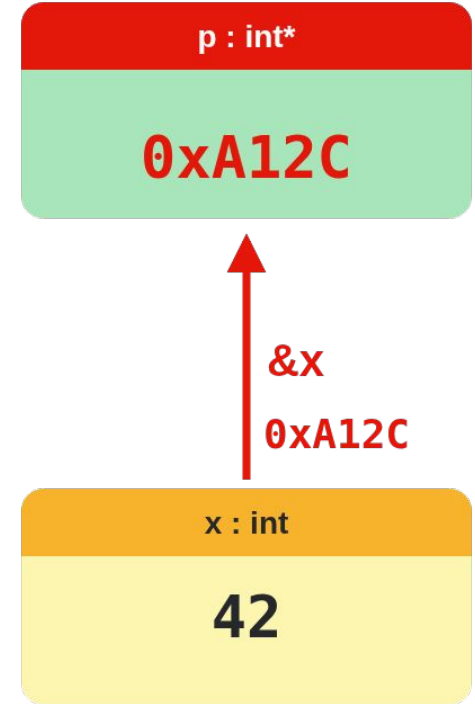
p : int*

0xA12C

Pointers

- Declared with (type) *
- Get a variable's address with &

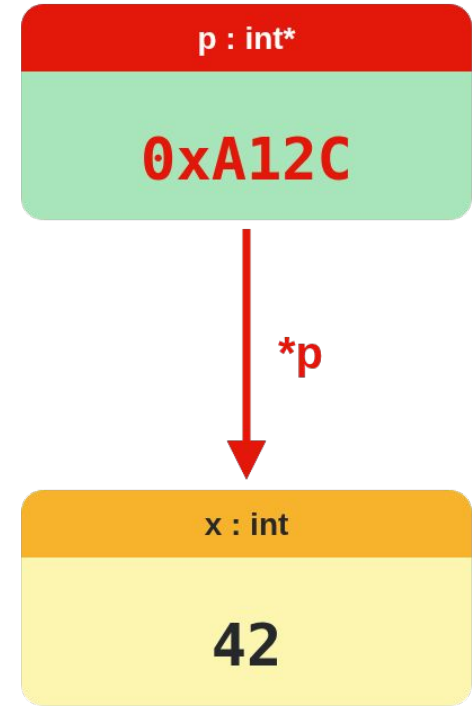
```
int * p;  
p = &x;
```



Pointers

- Declared with (type) *
- Get a variable's address with &
- Get a pointer's value with *

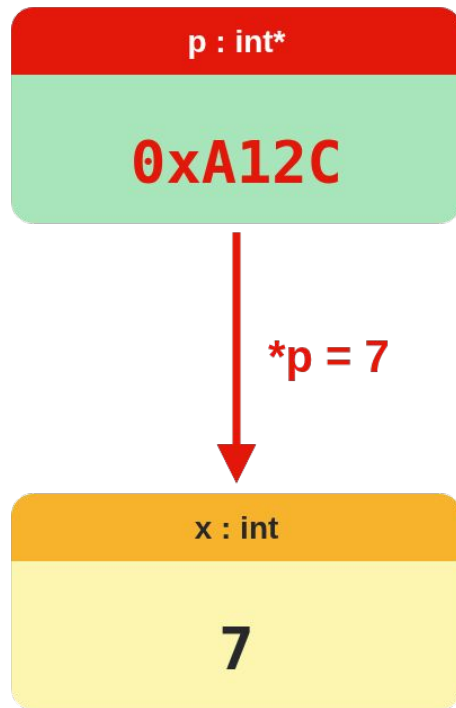
```
int * p;  
p = &x;  
(*p == 42) -> True
```



Pointers

- Declared with (type) *
- Get a variable's address with &
- Get a pointer's value with *
 - Can modify the value, too

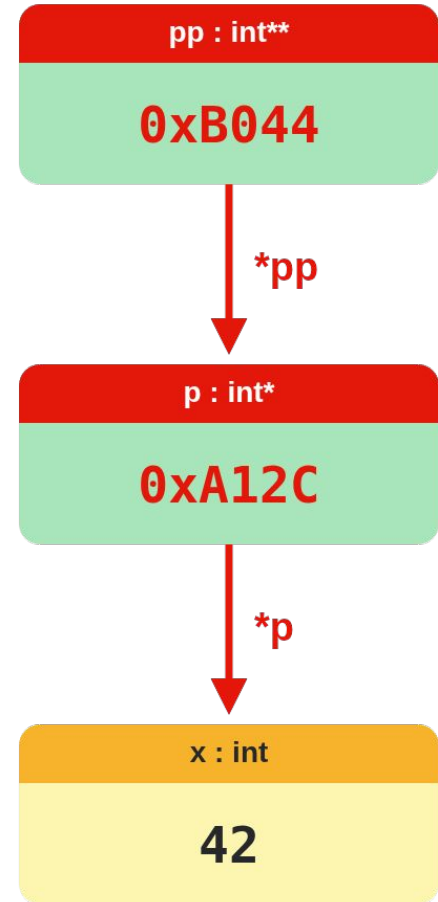
```
int * p;  
p = &x;  
(*p == 42) -> True  
*p = 7;
```



Pointers

- Declared with (type) *
- Get a variable's address with &
- Get a pointer's value with *
 - Can modify the value, too
- Pointers to pointers

```
int * p;  
p = &x;  
(*p == 42) -> True  
*p = 7;  
int ** p; /* Arrays! */
```



Arrays

- Declare with length for auto-allocate
 - Goes on stack, not heap (no malloc!)
 - GLOBAL goes into the text section of program memory
- Variable (a) is the address of the 0th element
- Index at 0, +1 for each element
 - Auto-adds correct span based on array type

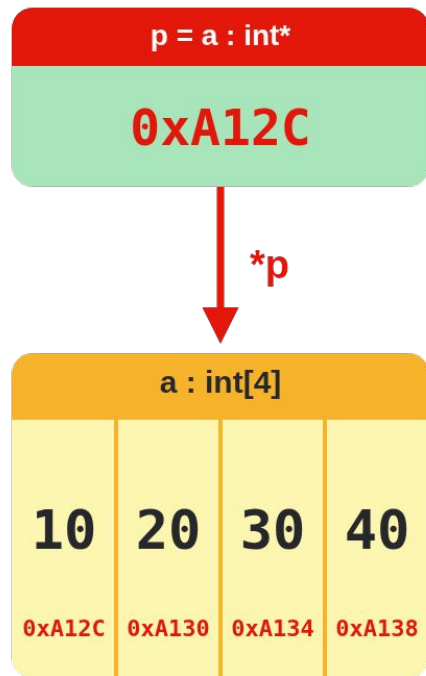
a : int[4]			
10	20	30	40
0xA12C	0xA130	0xA134	0xA138

&a[0] == a

a[2] == 30

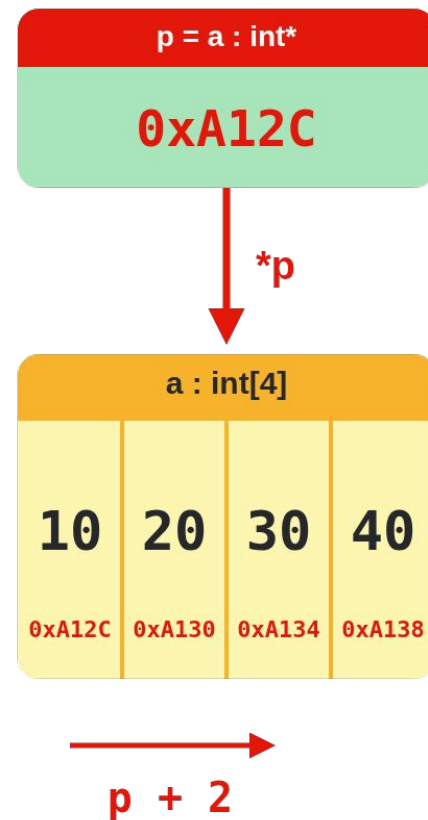
Arrays

- Declare with length for auto-allocate
 - Goes on stack, not heap (no malloc!)
 - GLOBAL goes into the text section of program memory
- Variable (a) is the address of the 0th element
- Index at 0, +1 for each element
 - Auto-adds correct span based on array type
- Pointer to array behaves the same



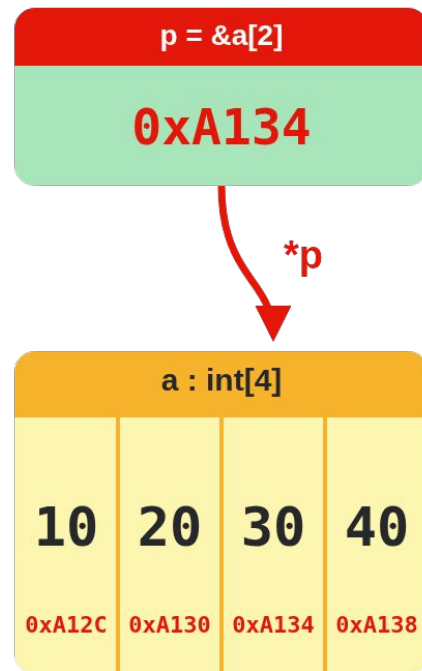
Arrays

- Declare with length for auto-allocate
 - Goes on stack, not heap (no malloc!)
 - GLOBAL goes into the text section of program memory
- Variable (a) is the address of the 0th element
- Index at 0, +1 for each element
 - Auto-adds correct span based on array type
- Pointer to array behaves the same
 - Including auto-adding correct span!



Arrays

- Declare with length for auto-allocate
 - Goes on stack, not heap (no malloc!)
 - GLOBAL goes into the text section of program memory
- Variable (a) is the address of the 0th element
- Index at 0, +1 for each element
 - Auto-adds correct span based on array type
- Pointer to array behaves the same
 - Including auto-adding correct span!



`*p == p[0] == 30`

Tools and Links

- gdb: breakpoints, backtraces, inspecting variables
- Valgrind: finds memory errors and leaks, runs slowly
- Modern C, Jens Gustedt (free): gustedt.gitlabpages.inria.fr/modern-c
- Beej's Guide to C: beej.us/guide/bgc
- Compiler Explorer: godbolt.org

OpenMP Next

A lot of this material is echoed by or comes from from an excellent tutorial by LLNL:
<https://hpc-tutorials.llnl.gov/openmp/introduction/#openmp-is>

OpenMP: Intro

- Open Multi-Processing
- API for shared-memory parallel programming in C, C++, and Fortran
- Consists of compiler directives (`#pragma omp`) and env variables, plus a small runtime library
- Fork-join model: one thread runs, forks a team, then joins back
- Easy to use, easy to debug, not guaranteed to be the fastest

```

#include <omp.h>

main () {

    int nthreads, tid;

    /* Fork a team of threads with each thread having a private tid variable */
    #pragma omp parallel private(tid)
    {

        /* Obtain and print thread id */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0)
        {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    } /* All threads join master thread and terminate */

}

```

Compiler / Platform	Compiler	Flag
Intel Linux Opteron/Xeon	icc icpc ifort	-qopenmp
PGI Linux Opteron/Xeon	pgcc pgCC pgf77 pgf90	-mp
GNU Linux Opteron/Xeon IBM Blue Gene	gcc g++ g77 gfortran	-fopenmp
IBM Coral Systems	xlc_r, cc_r xlc_r, xlc++_r xlc89_r xlc99_r xlf_r xlf90_r xlf95_r xlf2003_r *Be sure to use a thread-safe compiler - its name ends with _r	- qsmp=omp

Figures from https://hpc-tutorials.llnl.gov/openmp/programming_model/#shared-memory-model

From LLNL Tutorial: OpenMP is not:

- Necessarily implemented identically by all vendors
- Guaranteed to make the most efficient use of shared memory
- Required to check for data dependencies, data conflicts, race conditions, or deadlocks
- Required to check for code sequences that cause a program to be classified as non-conforming
- Designed to guarantee that input or output to the same file is synchronous when executed in parallel. The programmer is responsible for synchronizing input and output.

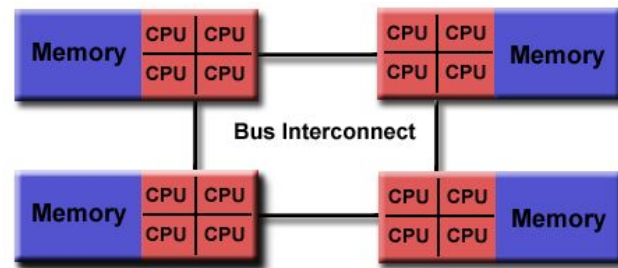
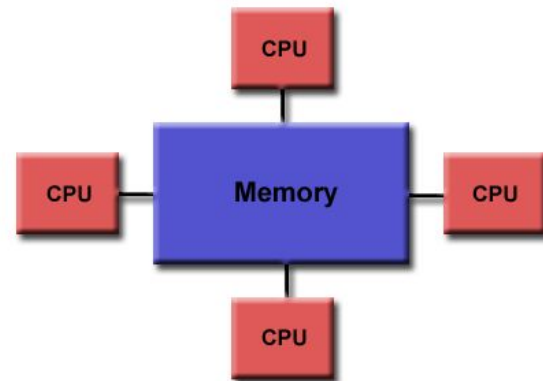
<https://hpc-tutorials.llnl.gov/openmp/introduction/#openmp-is>

Memory and Execution

- Uses threads (not processes)
- Means:
 - Quick
 - Easy to communicate
 - Memory is shared, UMA(top) or NUMA(bottom)
 - More cores? More threads!
 - Oversubscription hurts performance

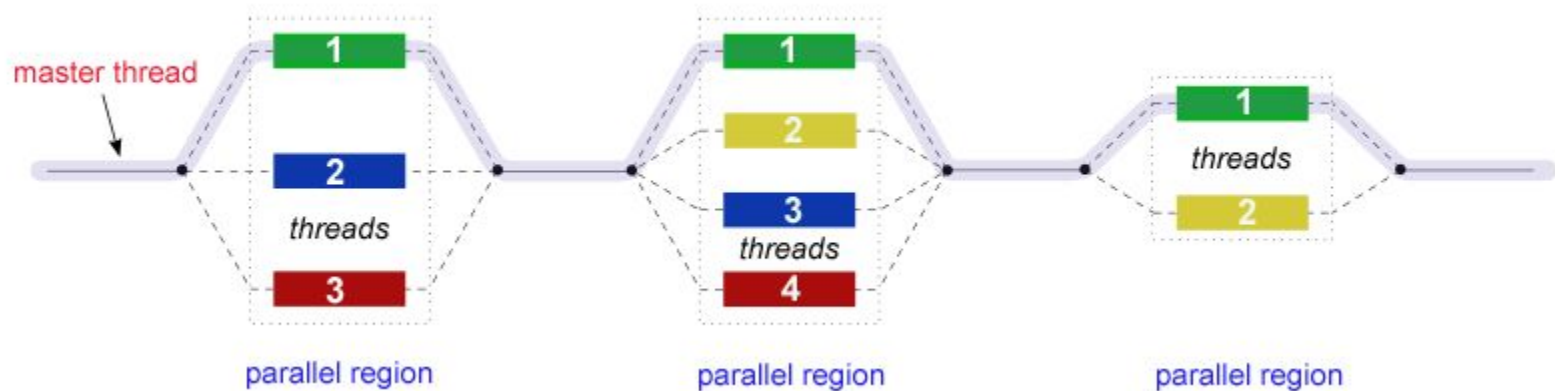
Figures from

https://hpc-tutorials.llnl.gov/openmp/programming_model/#shared-memory-model



Memory and Execution

- Fork-join model
- Threads come from a worker pool of OS threads in the OpenMP runtime



Figures from https://hpc-tutorials.llnl.gov/openmp/programming_model/#shared-memory-model

Compiler Directives

- sentinel directive-name [clause, ...]
- Goes above a block
- Used to:
 - Indicate a parallel region
 - Serialize other sections
 - Synchronize
 - And more!

#pragma omp	directive-name	[clause, ...]	newline
Required for all OpenMP C/C++ directives.	A valid OpenMP directive must appear after the pragma and before any clauses.	Optional. Clauses can be in any order, and repeated as necessary unless otherwise restricted.	Required. Precedes the structured block which is enclosed by this directive.

Figures from https://hpc-tutorials.llnl.gov/openmp/programming_model/#shared-memory-model

Runtime Library Routines

- Look like functions
- `#include <omp.h>`
- Used to:
 - Set/query the number of threads
 - Identify a thread
 - Initialize, set, and terminate locks
 - Query time

Runtime Library Routines

- `void omp_set_num_threads(int n);`
 - sets the team size for later parallel regions
- `int omp_get_num_threads(void);`
 - returns the size of the current team. It returns 1 outside a parallel region
- `int omp_get_max_threads(void);`
 - returns the largest team a new parallel region would get. Use this outside a region, for example to size a per-thread array
- `int omp_get_num_procs(void);`
 - returns the number of processors available to the program
- `int omp_in_parallel(void);`
 - returns nonzero if called inside an active parallel region
- `int omp_get_thread_num(void);`
 - returns this thread's number within its team, from 0 to team size minus 1

Environment Variables

- Many are available
- Used to:
 - Set the number of threads
 - Bind threads to procs
 - Indicate loop iteration division strategies
 - Control nested parallelism
 - Much more...

```

#include <omp.h>

main () {

    int nthreads, tid;

    /* Fork a team of threads with each thread having a private tid variable */
    #pragma omp parallel private(tid)
    {

        /* Obtain and print thread id */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0)
        {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    } /* All threads join master thread and terminate */

}

```

Compiler Directives

Examples of structured blocks of code:

```
#pragma omp parallel
```

```
{
```

```
    id = omp_thread_num();
```

```
    Array[id] = some_function(id);
```

```
}
```

```
#pragma omp for
```

```
for (int i = 0; i < N; i++){
```

```
    Array[i] = some_function(i);
```

```
}
```

Compiler Directive: Parallel

- Indicates one version of the code should be started for every worker available
- In opposition to “serial”
- Problems in the below code?

```
#pragma omp parallel
{
    int num_threads = omp_get_num_threads();
    int thread_id = omp_get_thread_num();
    printf("Hello from thread %d out of
           %d\n", thread_id, num_threads);
}
```

```
int num_threads, thread_id;

#pragma omp parallel
{
    num_threads = omp_get_num_threads();
    thread_id = omp_get_thread_num();
    printf("Hello from thread %d out of
           %d\n", thread_id, num_threads);
}
```

Compiler Directives

- Clause: Private and Firstprivate
 - Both indicate copies of a variable should go to the thread
 - Firstprivate also carries over the initialized value

#pragma omp	directive-name	[clause, ...]	newline
Required for all OpenMP C/C++ directives.	A valid OpenMP directive must appear after the pragma and before any clauses.	Optional. Clauses can be in any order, and repeated as necessary unless otherwise restricted.	Required. Precedes the structured block which is enclosed by this directive.

Figures from https://hpc-tutorials.llnl.gov/openmp/programming_model/#shared-memory-model

Compiler Directive

- Clause: Private and Firstprivate
 - Both indicate copies of a variable should go to the thread
 - Firstprivate also carries over the initialized value

```
int initial_value = 15;

#pragma omp parallel private(initial_value)
{
    printf("Thread %d sees initial_value =
           %d\n", omp_get_thread_num(),
           initial_value);
}
```

```
int initial_value = 15;

#pragma omp parallel
firstprivate(initial_value)
{
    printf("Thread %d sees initial_value =
           %d\n", omp_get_thread_num(),
           initial_value);
}
```

Compiler Directive: For

- Indicates a for loop to break across multiple worker threads
- Note: outermost thread only

```
int num_threads, thread_id;

omp_set_num_threads(4);

#pragma omp parallel for private(num_threads, thread_id)
for (int i = 1; i <= 10; i++){
    num_threads = omp_get_num_threads();
    thread_id = omp_get_thread_num();
    printf("Hello from iteration %i from thread %d out of %d\n", i, thread_id,
          num_threads);
}
```


Compiler Directive: For

- Indicates a for loop to break across multiple worker threads
- Note: outermost thread only

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        ...
    }
}
```

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        ...
    }
}
```

Compiler Directives: Scheduling the work from blocks

- `schedule(static):`
 - Iterations split into equal chunks
- `schedule(dynamic, n):`
 - Threads take iterations at a time and get more as they finish
- `schedule(guided):`
 - Chunks start large and shrink
- Static has the least overhead and works well when loops will be similar
- Dynamic and guided help when iteration cost varies

Synchronization

- `#pragma omp master`
 - Run the following block only in thread 0
- `#pragma omp critical [ name ]`
 - Parallelism still possible, but locks around this region
 - Optional name makes it possible to have multiple critical regions
 - Careful, this can kill performance
- `#pragma omp barrier`
 - All threads go to this point and wait until all other threads in the team also reach this point
- `#pragma omp flush (list)`
 - Make sure the variables listed in (list) are written out to memory
- `#pragma omp ordered (for)`
 - Like critical, but also do these in the iterative order

Sections

- Another mechanism to break work apart
- `#pragma omp parallel sections`

```
#pragma omp parallel sections
{
    #pragma omp section
    {
        Do a task
    }

    #pragma omp section
    {
        Do another task
    }
}
```

Runtime Library Routines

- `omp_lock_t` is the lock type
- `void omp_init_lock(omp_lock_t *lock);`
 - Initializes the lock
- `void omp_set_lock(omp_lock_t *lock);`
 - Acquires the lock, blocking until available
- `void omp_unset_lock(omp_lock_t *lock);`
 - Releases the lock, can only be called by the holding thread
- `int omp_test_lock(omp_lock_t *lock);`
 - Tries to acquire without blocking, and returns nonzero if it got the lock.
- `void omp_destroy_lock(omp_lock_t *lock);`
 - Variable tidiness

Runtime Library Routines

- `double omp_get_wtime(void);`
 - Returns wall-clock time in seconds from a measured point that will be consistent across the system
 - Use differences in calls to this to time around OpenMP calls
- `double omp_get_wtick(void);`
 - Returns the timer's resolution in seconds

Coming Up...

- Tonight: slides, more practice problems
- Next week: MPI (message passing)